Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Akhir
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

Pengenalan Mikrokontroler

Faruq Awliya Labiib - 5024241020

2026

1 Langkah-Langkah Percobaan

Percobaan 1: Pengenalan STM32 Blackpill (Blink)

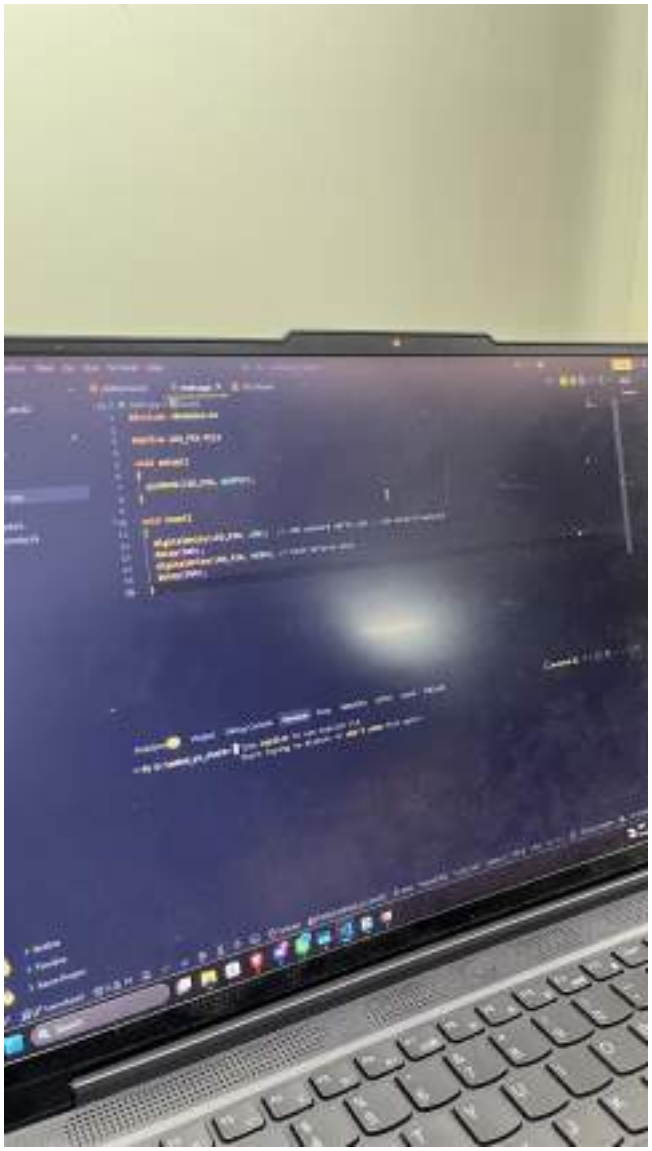
1. Pastikan project Modul 1 sudah dibuat dan isi platformio.ini disesuaikan untuk board STM32 Blackpill dengan framework Arduino dan upload protocol stlink.

2. Hubungkan ST-Link ke STM32 Blackpill menggunakan kabel jumper (SWDIO ke SWDIO, SWCLK ke SWCLK, GND ke GND, 3.3V ke 3V3), lalu colokkan ST-Link ke komputer via USB.



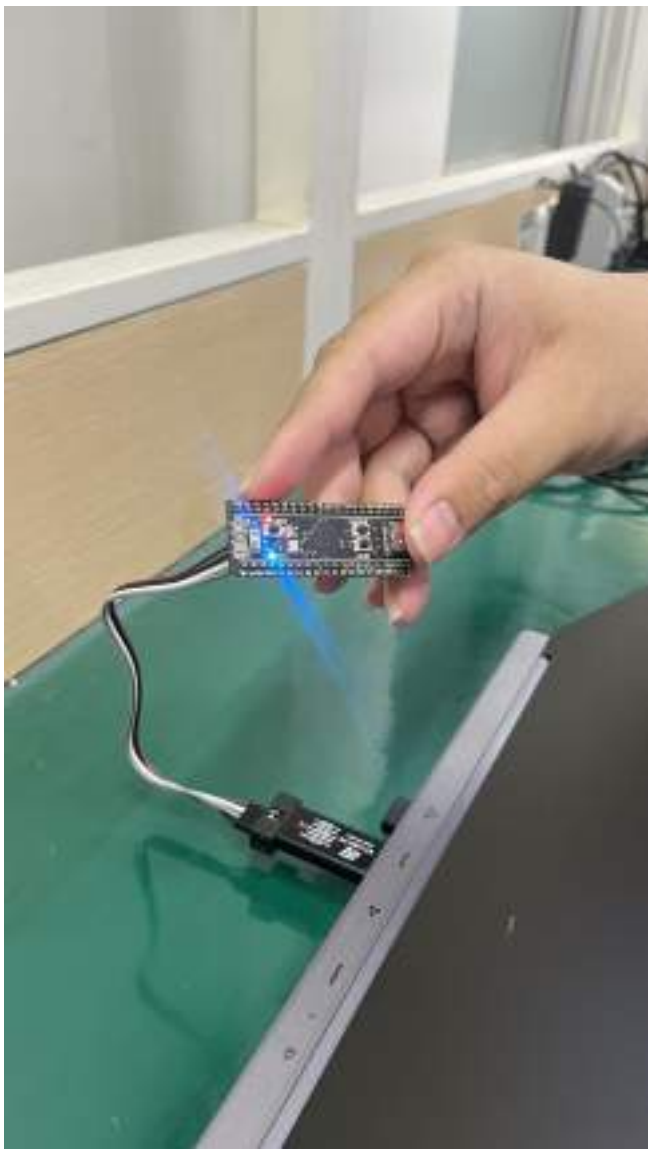
Gambar 1: Pengkabelan ST-Link ke STM32 Blackpill

3. Tulis kode program Blink STM32 Arduino pada file src/main.cpp, lalu lakukan proses Build dan Upload via PlatformIO.



Gambar 2: Proses Upload Program Blink ke STM32

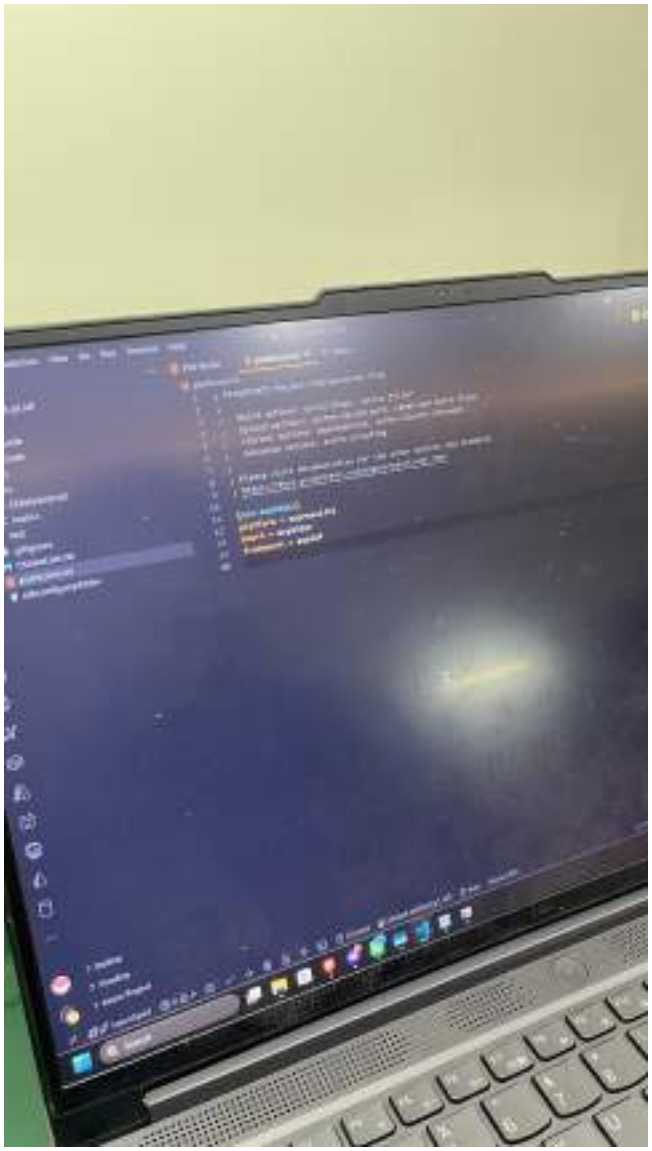
4. Amati indikator LED onboard di pin PC13 pada board STM32 Blackpill, pastikan berkedip dengan interval menyala setengah detik dan mati setengah detik secara berulang.



Gambar 3: Pengujian Hasil Kedipan LED Onboard PC13

Percobaan 2: Pengenalan ESP32 dengan Framework ESP-IDF (Blink)

1. Ganti isi file platformio.ini dengan konfigurasi untuk board esp32dev dan framework espidf.



Gambar 4: Konfigurasi platformio.ini Framework ESP-IDF

2. Hubungkan board ESP32 DevKit ke komputer menggunakan kabel USB dan pastikan port COM terdeteksi pada sistem.

3. Hapus file src/main.cpp, buat file baru bernama src/main.c, masukkan kode program Blink ESP-IDF berbasis app_main, lalu jalankan perintah Build dan Upload.



Gambar 5: Penulisan dan Flashing Kode main.c ESP-IDF

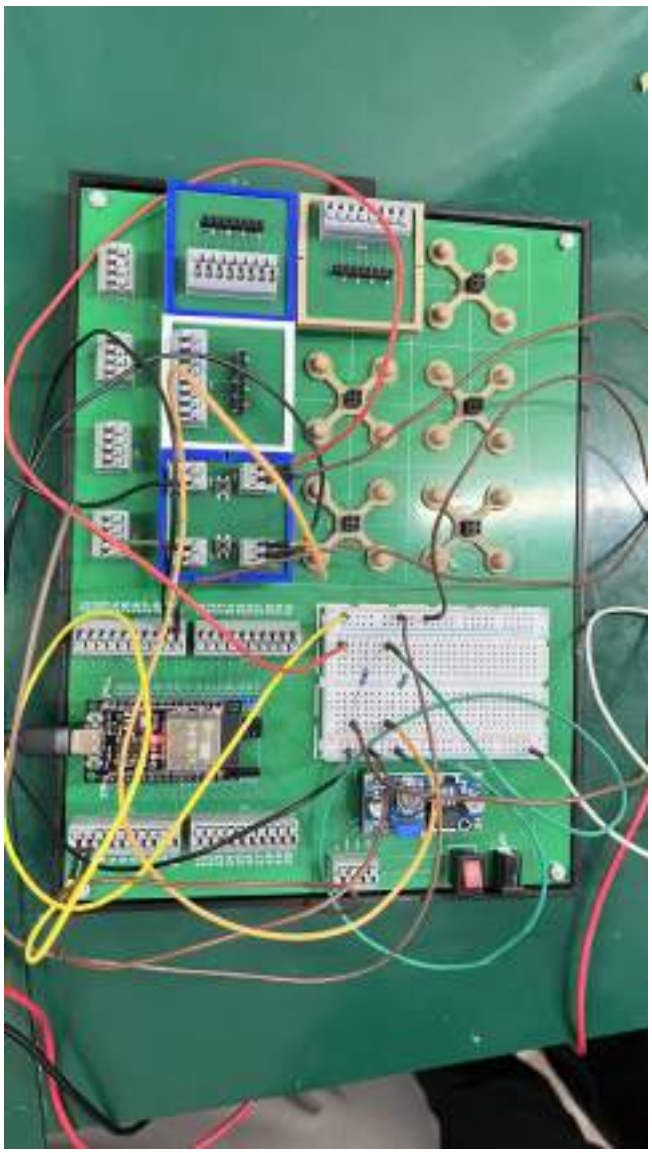
4. Amati hasil kedipan LED pada GPIO 2 dan lakukan analisis perbedaan struktur pemrograman antara framework ESP-IDF berbasis task FreeRTOS dan framework Arduino.



Gambar 6: Pengujian Kedipan LED Onboard ESP32

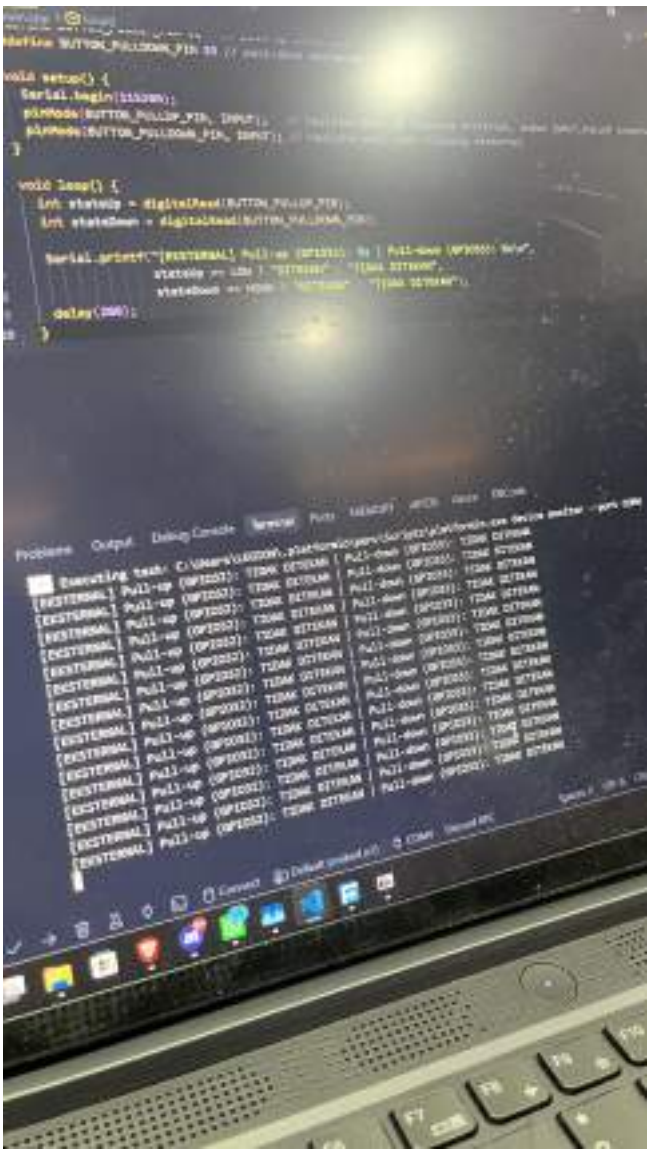
Percobaan 3: Pull-up dan Pull-down: Eksternal vs Internal (ESP32 + Framework Arduino)

1. Ubah kembali isi platformio.ini ke framework Arduino, hapus src/main.c, dan buat ulang file src/main.cpp.
2. Rangkai dua pushbutton pada breadboard sesuai Skema Bagian A: Pushbutton 1 pada GPIO 32 dengan resistor pull-up eksternal 10 kOhm ke 3.3V, dan Pushbutton 2 pada GPIO 33 dengan resistor pull-down eksternal 10 kOhm ke GND.



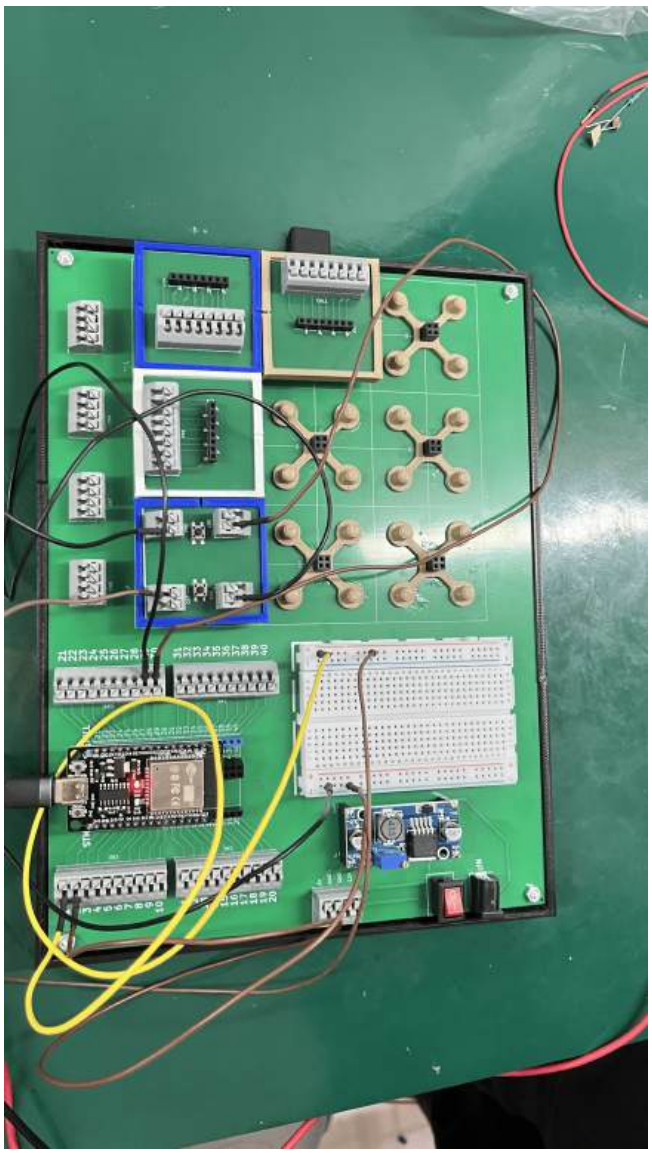
Gambar 7: Rangkaian Pull-up dan Pull-down Eksternal

3. Upload Kode Program A (Eksternal), buka Serial Monitor pada baud rate 115200, lalu tekan kedua tombol secara bergantian dan catat pembacaan log statusnya.



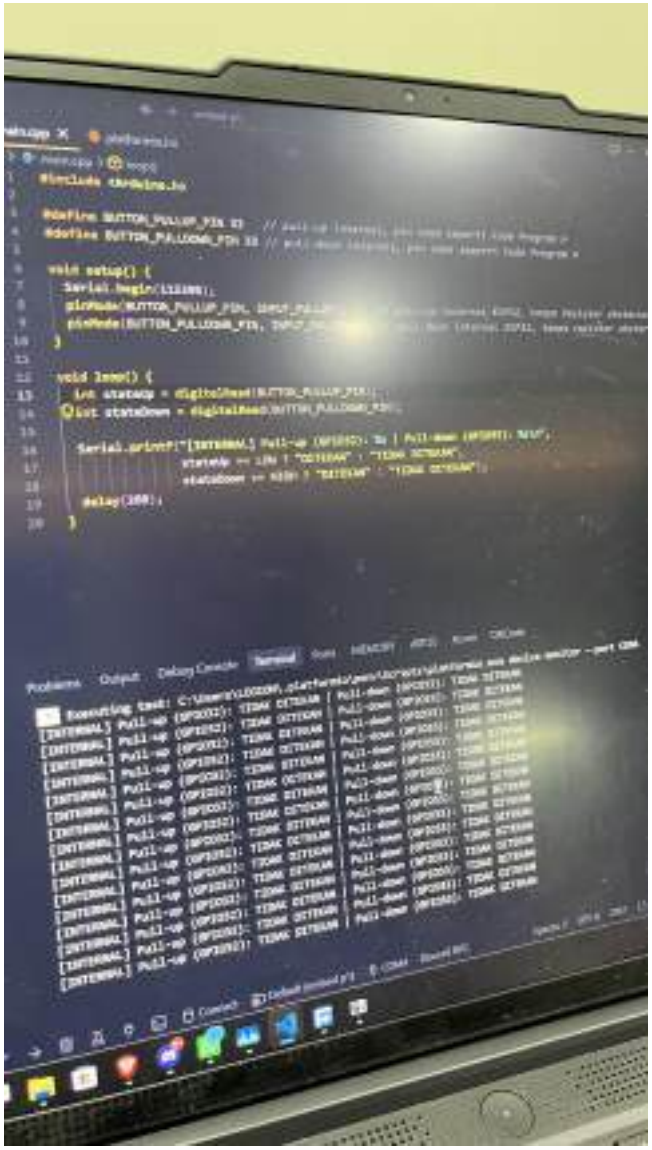
Gambar 8: Pengujian Pembacaan Serial Resistor Eksternal

4. Lepaskan kedua resistor eksternal dari breadboard sesuai Skema Bagian B, sementara kedua pushbutton tetap terhubung ke GPIO 32 dan GPIO 33.



Gambar 9: Pelepasan Resistor Eksternal dari Rangkaian

5. Ganti kode pada src/main.cpp dengan Kode Program B (Internal) yang menggunakan mode INPUT_PULLUP dan INPUT_PULLDOWN, kemudian lakukan Build dan Upload.



Gambar 10: Flashing Program Pull-up/Pull-down Internal

6. Tekan kembali kedua tombol dan amati pembacaan status logika pada Serial Monitor.



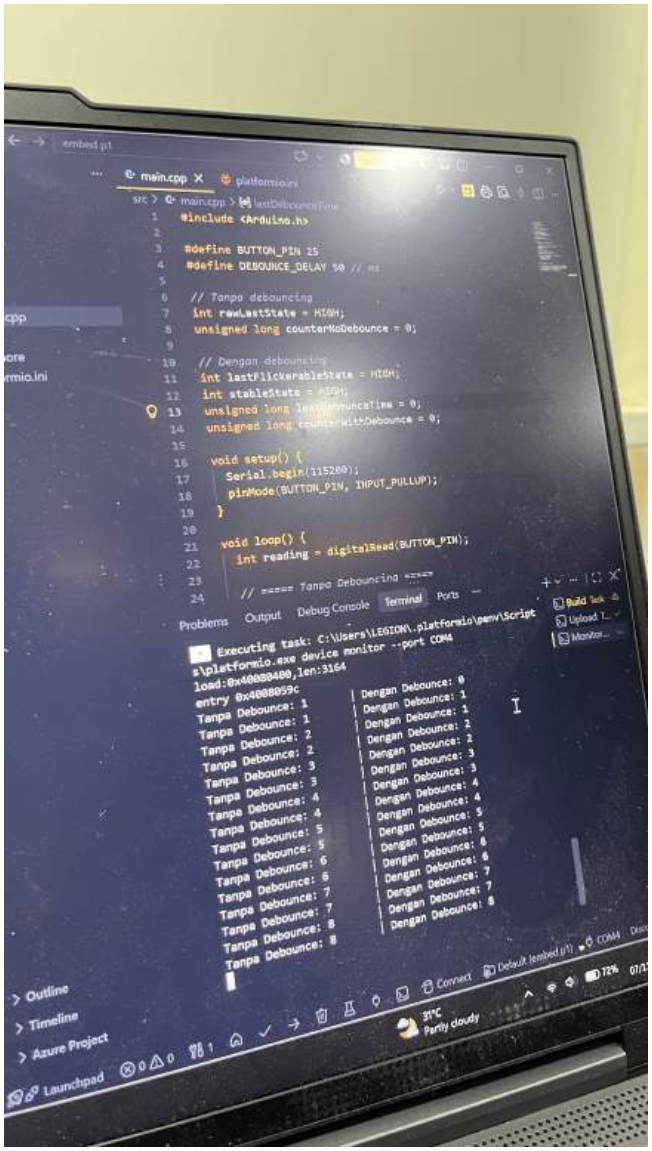
Gambar 11: Pengujian Pembacaan Serial Resistor Internal

7. Bandingkan logika pembacaan antara konfigurasi eksternal dan internal untuk memastikan hasil pembacaan logika tombol bernilai konsisten.

Percobaan 4: Debouncing pada Input Tombol GPIO (ESP32 + Framework Arduino)

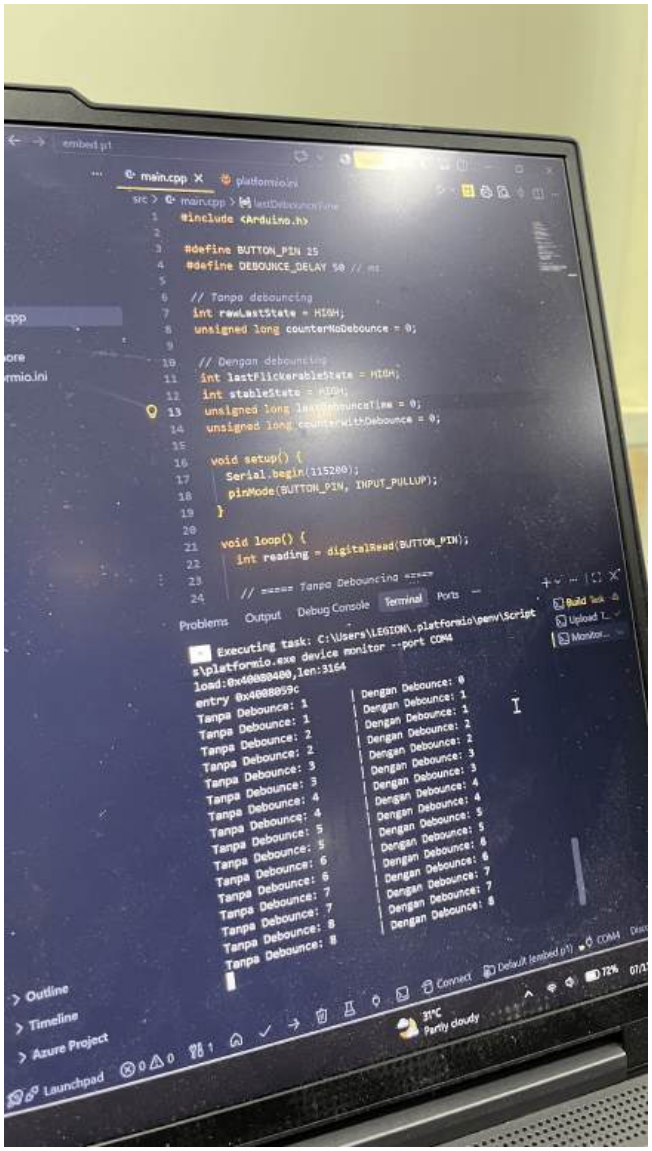
1. Rangkai satu pushbutton pada GPIO 25 ke GND menggunakan konfigurasi pull-up internal ESP32.

2. Tulis program perbandingan pencacah tanpa debouncing dan dengan debouncing pada file src/main.cpp, lalu lakukan Build dan Upload.



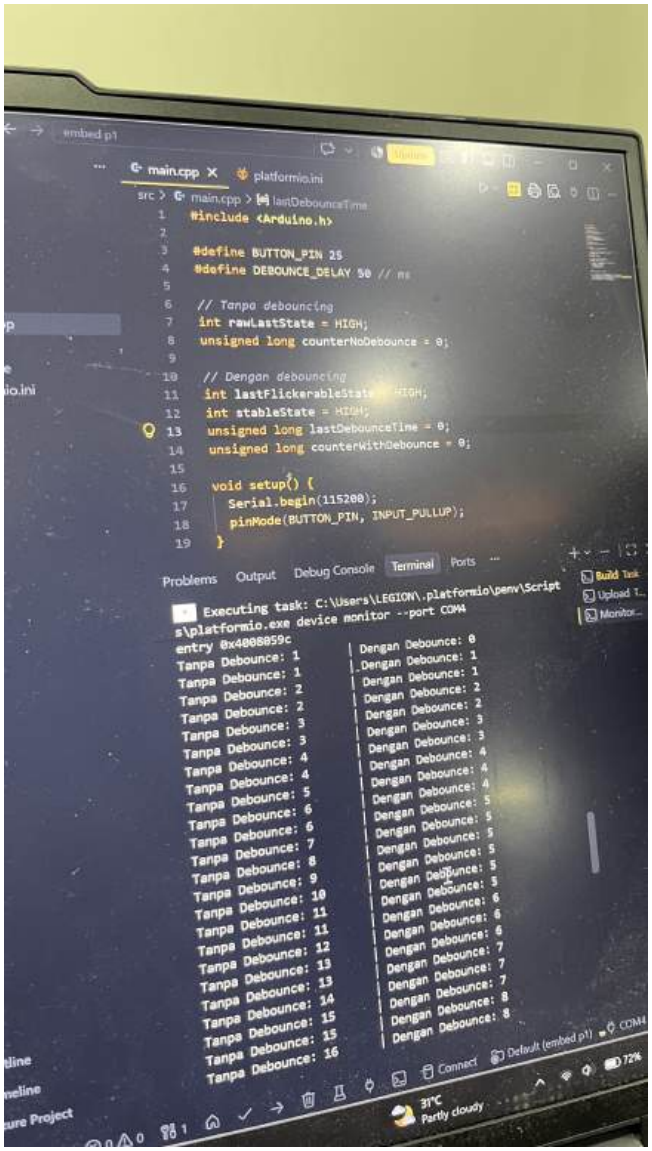
Gambar 12: upload Program Perhitungan Debouncing

3. Buka Serial Monitor, tekan tombol satu kali secara normal, lalu amati serta bandingkan pertambahan nilai counterNoDebounce dan counterWithDebounce.



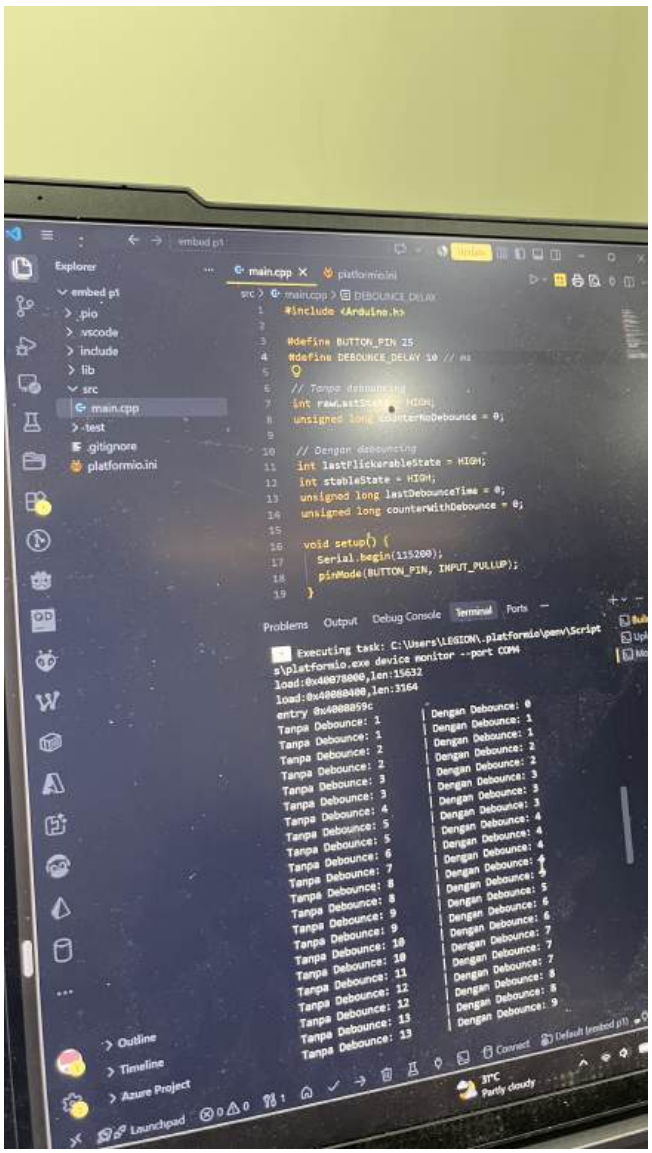
Gambar 13: Pengamatan Bouncing pada Serial Monitor

4. Ulangi penekanan tombol dengan bervariasi kecepatan (tekan cepat dan lambat) untuk melihat kemunculan fenomena getaran mekanis tombol.



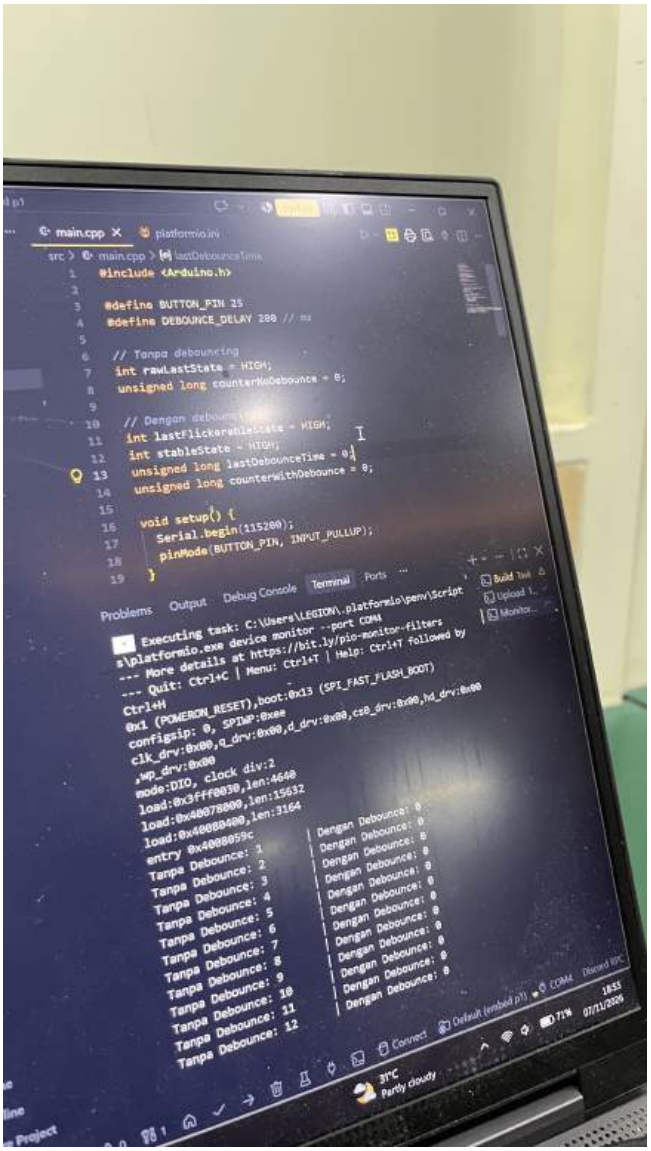
Gambar 14: Pengujian Penekanan Tombol Cepat dan Lambat

5. Ubah nilai konstanta DEBOUNCE_DELAY menjadi 10 ms pada kode program, re-upload, lalu uji kembali penekanan tombol.



Gambar 15: Pengujian dengan Nilai Debounce Delay 10 ms

6. Ubah nilai konstanta DEBOUNCE_DELAY menjadi 200 ms, re-upload, dan lakukan penekanan tombol secara cepat berulang kali.

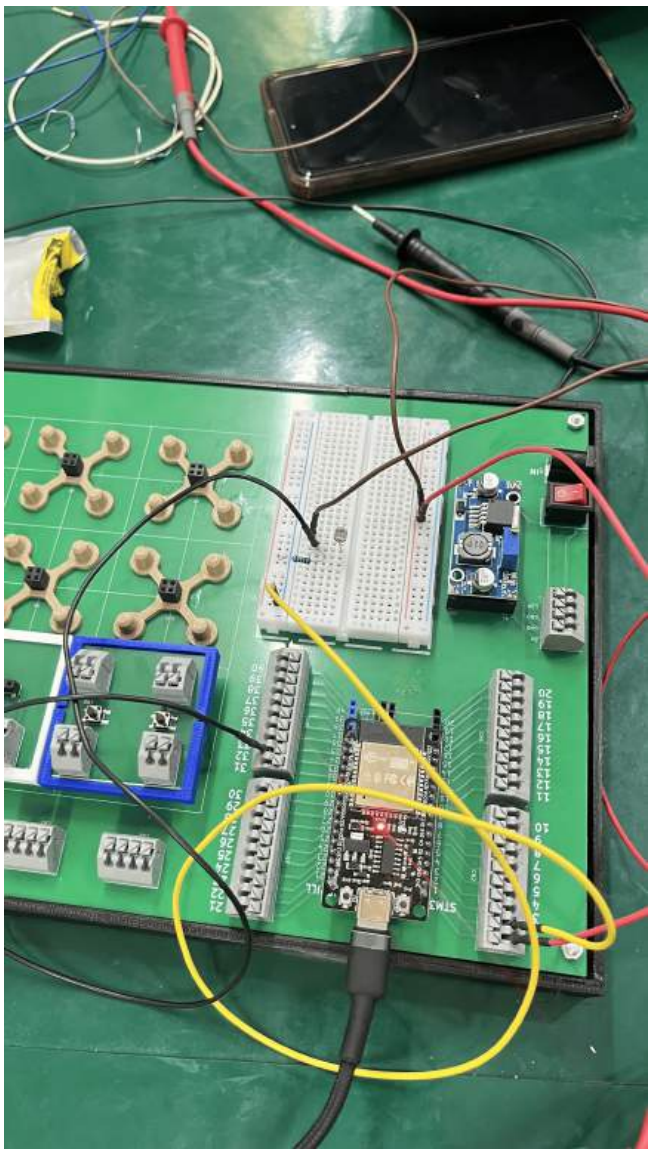


Gambar 16: Pengujian dengan Nilai Debounce Delay 200 ms

7. Analisis perbandingan kestabilan sinyal dan responsivitas pembacaan tombol dari ketiga variasi waktu debounce (10 ms, 50 ms, dan 200 ms).

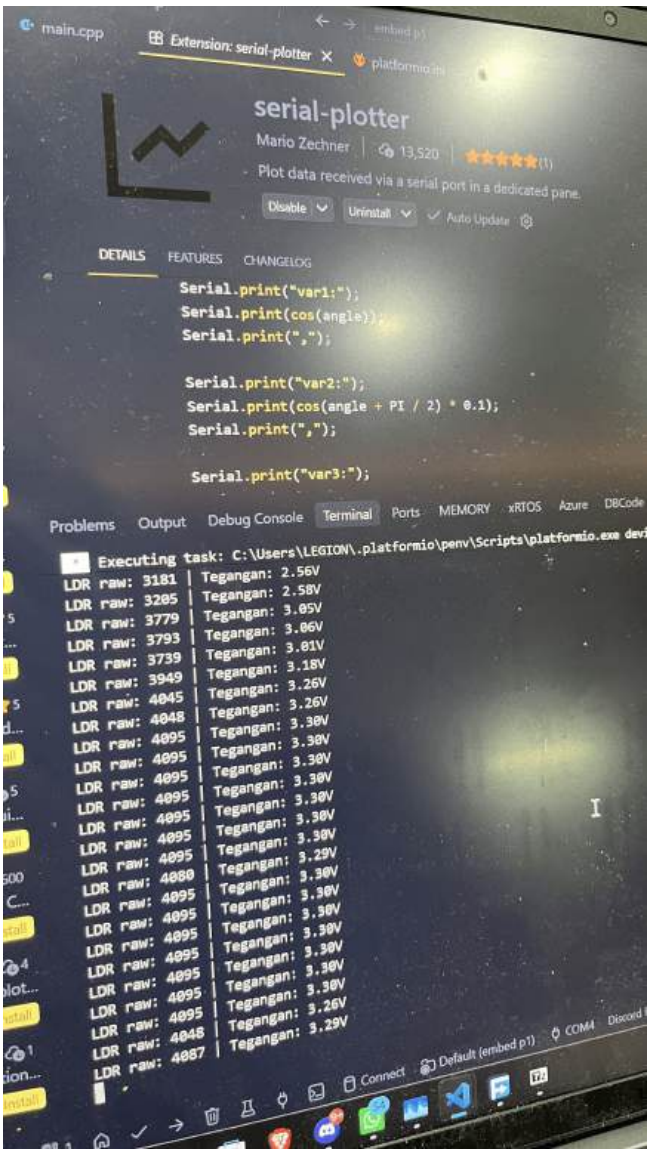
Percobaan 5: Akses GPIO Analog: Pembacaan ADC dengan LDR (ESP32 + Framework Arduino)

1. Rangkai sensor LDR dan resistor 10 kOhm sebagai pembagi tegangan (LDR ke 3.3V, resistor ke GND, dan titik tengah pembagi tegangan dihubungkan ke GPIO 34).



Gambar 17: Rangkaian Pembagi Tegangan LDR pada GPIO 34

2. Tulis program pembacaan nilai ADC 12-bit dan konversi ke satuan tegangan volt pada src/main.cpp.
3. Lakukan proses Build dan Upload, kemudian buka Serial Monitor pada kecepatan 115200 baud.
4. Tutup permukaan LDR dengan tangan untuk kondisi gelap, lalu beri pencahayaan langsung dengan lampu untuk kondisi terang, dan amati perubahan nilai ADC mentah (0 hingga 4095) beserta nilai tegangannya.



Gambar 18: Pengujian Respon LDR Terhadap Gelap dan Terang

5. Analisis bagaimana GPIO mode analog input (ADC) dapat menyajikan kontinuum nilai tegangan, berbeda dari logika biner pada mode digital input.

2 Analisis Hasil Percobaan

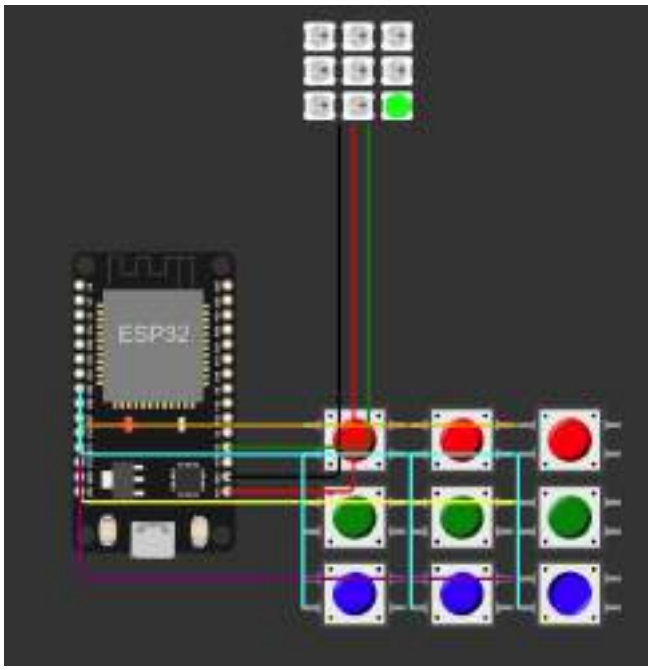
Pada Percobaan 1 dan Percobaan 2, dilakukan pengenalan mikrokontroler STM32 Blackpill dan ESP32 beserta pemrosesan firmware menggunakan framework yang berbeda. Pada STM32 Blackpill, upload program dilakukan menggunakan ST-Link via protokol SWD dengan framework Arduino, di mana abstraksi hardware mempermudah kontrol pin seperti LED onboard PC13 (aktif LOW) melalui fungsi standar `pinMode`, `digitalWrite`, dan `delay`. Sementara itu, Percobaan 2 memanfaatkan ESP32 berbasis framework native ESP-IDF dengan struktur berbasis task FreeRTOS yang diawali dari fungsi `app_main`. Pemrograman ESP-IDF memberikan akses low-level melalui fungsi `gpio_set_direction`, `gpio_set_level`, dan delay non-blocking `vTaskDelay`, sehingga memperlihatkan perbedaan mendasar antara framework Arduino yang mengutamakan kemudahan dan ESP-IDF yang menawarkan kontrol hardware.

Percobaan 3 menganalisis perbedaan implementasi resistor pull-up dan pull-down secara eksternal maupun internal pada ESP32 untuk mencegah kondisi floating pada input digital GPIO. Saat tombol tidak ditekan, pin tanpa resistor pull akan berada pada kondisi logika yang tidak pasti akibat gangguan noise elektrikal. Pada konfigurasi pull-up, pin ditarik ke tegangan VCC (3.3V) sehingga bernilai HIGH saat terbuka dan LOW saat ditekan ke GND. Sebaliknya, pada konfigurasi pull-down, pin ditarik ke GND sehingga bernilai LOW saat terbuka dan HIGH saat ditekan ke VCC. Pengujian membuktikan bahwa penggunaan resistor internal melalui mode `INPUT_PULLUP` dan `INPUT_PULLDOWN` menghasilkan logika pembacaan yang persis sama dengan penggunaan resistor eksternal 10 kOhm, sehingga dapat menyederhanakan tata letak komponen fisik pada breadboard selama pin GPIO yang digunakan mendukung fitur pull internal tersebut.

Percobaan 4 mengevaluasi permasalahan bouncing pada tombol serta teknik debouncing berbasis software. Kontak fisik pada tombol saat ditekan maupun dilepas selalu menghasilkan sinyal HIGH-LOW berulang dalam hitungan milidetik, yang menyebabkan variabel `counterNoDebounce` bertambah lebih dari satu kali hanya untuk satu kali penekanan tombol. Dengan mengimplementasikan algoritma debouncing menggunakan fungsi `millis()` dan batasan waktu `DEBOUNCE_DELAY`, perubahan sinyal baru dianggap valid setelah mencapai kondisi stabil selama periode waktu tertentu, sehingga `counterWithDebounce` hanya bertambah tepat satu kali per penekanan. Variasi nilai jeda menunjukkan trade-off di mana nilai delay yang terlalu kecil (10 ms) masih berpotensi meloloskan noise bouncing, sedangkan nilai delay yang terlalu besar (200 ms) menyebabkan responsivitas tombol terasa lambat dan kurang presisi saat ditekan berulang secara cepat.

Percobaan 5 menguji mode akses GPIO ketiga yaitu analog input memanfaatkan Analog-to-Digital Converter (ADC) 12-bit pada ESP32. Rangkaian pembagi tegangan antara sensor LDR dan resistor 10 kOhm pada GPIO 34 mengubah variasi resistansi LDR akibat intensitas cahaya menjadi perubahan tegangan analog berkisar antara 0 hingga 3.3V. Fungsi analogRead memetakan tegangan kontinu tersebut menjadi nilai digital mentah 0 sampai 4095. Berbeda dengan digital input yang hanya mampu membedakan dua kondisi biner (HIGH atau LOW), mode analog input memungkinkan sistem mengukur dan merepresentasikan berbagai tingkat gradasi kondisi lingkungan secara kontinu, seperti perbedaan kondisi gelap, redup, hingga terang benderang.Terlihat saat ditutup oleh jari dan ketika disenteri dari HP, ketika ditutup jari nilainya 1.12V dan ketika disenteri nilainya 3.3V

3 Hasil Tugas Modul



Gambar 19: Screenshot Simulasi Wokwi

Berikut Source code main.cpp

```
#include <Arduino.h>
#include <Adafruit_NeoPixel.h>

#define PIN_NEOPIXEL 2
#define NUM_PIXELS 9

Adafruit_NeoPixel pixels(NUM_PIXELS, PIN_NEOPIXEL, NEO_GRB + NEO_KHZ800);

const int rowPins[3] = {13, 12, 14}; // Pin Baris (Output)
const int colPins[3] = {27, 26, 25}; // Pin Kolom (Input Pull-up)

int lastFlickerableState[9];
int stableState[9];
unsigned long lastDebounceTime[9];
const unsigned long debounceDelay = 50;

void setup() {
  Serial.begin(115200);

  pixels.begin();
  pixels.clear();
  pixels.show();
```

```
  for (int i = 0; i < 3; i++) {
    pinMode(rowPins[i], OUTPUT);
    digitalWrite(rowPins[i], HIGH);
  }

  for (int i = 0; i < 3; i++) {
    pinMode(colPins[i], INPUT_PULLUP);
  }

  for (int i = 0; i < 9; i++) {
    lastFlickerableState[i] = HIGH;
    stableState[i] = HIGH;
    lastDebounceTime[i] = 0;
  }

}

void loop() {
  for (int r = 0; r < 3; r++) {

    digitalWrite(rowPins[r], LOW);

    delay(2);

    for (int c = 0; c < 3; c++) {
      int index = (r * 3) + c;
      int reading = digitalRead(colPins[c]);

      if (reading != lastFlickerableState[index]) {
        lastDebounceTime[index] = millis();
      }

      if ((millis() - lastDebounceTime[index]) > debounceDelay) {
        if (reading != stableState[index]) {
          stableState[index] = reading;

          if (stableState[index] == LOW) {
            Serial.printf("PRESSED: Rows %d, Cols %d (LED %d)\n", r+1, c+1, index);
            pixels.setPixelColor(index, pixels.Color(0, 255, 0));
          } else {
            Serial.printf("RELEASED: Rows %d, Cols %d (LED %d)\n", r+1, c+1, index);
            pixels.setPixelColor(index, pixels.Color(0, 0, 0));
          }
          pixels.show();
```

```
        }
      }
      lastFlickerableState[index] = reading;
    }

    digitalWrite(rowPins[r], HIGH);
  }
}
```

1. Debouncing tetap diperlukan karena fenomena bouncing murni disebabkan oleh sifat mekanis tombol yang bergetar saat bersentuhan atau terlepas, terlepas dari berapa banyak tombol yang ditekan. Tanpa debouncing, satu kali tekanan fisik pada matriks akan terbaca oleh mikrokontroler sebagai transisi sinyal HIGH-LOW berulang yang memicu respons berganda secara tidak valid.

2. Pada rancangan pembacaan kolom ini, digunakan konfigurasi pull-up internal yang membuat pin selalu membaca kondisi HIGH saat tombol tidak ditekan. Kondisi pin dilarang dibiarkan floating karena level tegangannya akan menjadi tidak pasti dan sangat rentan terhadap noise, sehingga bisa terbaca HIGH atau LOW secara acak.

3. Perbedaan mendasarnya terletak pada pembacaan digital input yang hanya mendeteksi dua kondisi (HIGH atau LOW), sementara analog input membaca variasi tegangan yang terus berubah. Perangkat ADC mampu merepresentasikan banyak kondisi karena secara internal mengubah level tegangan kontinu tersebut menjadi nilai digital, seperti rentang 0 hingga 4095 pada ESP32.

4 Kesimpulan

Secara keseluruhan, Praktikum Modul 1 ini memberikan pemahaman dasar mengenai dasar pemrograman mikrokontroler (STM32 dan ESP32) serta framewoek yang berbeda (Arduino dan ESP-IDF). Melalui serangkaian percobaan, praktikan tidak hanya mampu mengonfigurasi pin GPIO sebagai output digital, tetapi juga memahami esensi mode input digital dengan penanganan kondisi floating menggunakan resistor pull-up/pull-down dan mitigasi noise mekanis lewat algoritma debouncing. Selain itu, pengenalan input analog menggunakan ADC untuk membaca sinyal dari logika biner menjadi data kontinu, yang kesemuanya merupakan dasar dalam sistem embedded .